מעבדה 4: תזמון תהליכים - Scheduling

הקדמה ומטרות

מעבדה זו עוסקת באחד התפקידים הקריטיים של מערכת ההפעלה: תזמון תהליכים. כאשר ישנם מספר תהליכים במצב "Ready" ורק מעבד אחד, המערכת צריכה להחליט מי ירוץ ובאיזה סדר.

מטרות המעבדה:

- הבנת ההבדל בין תזמון עם הפקעה ולא הפקעה.
- מימוש סימולציות של אלגוריתמי תזמון נפוצים.
- חישוב מדדי ביצוע: זמן המתנה (Waiting Time) וזמן סבב (Turnaround Time).

מבנה הנתונים והקבצים

במעבדה זו נשתמש במבנה נתונים המייצג תהליך בסימולטור .עליכם להגדיר בשפת C, מבנה (struct) בשם $Process$ שיכיל:

- $int\ id$ מזהה התהליך.
- $int\ arrival_time$ זמן הגעה לתור המוכנים (Ready)
- $int\ burst_time$ זמן הביצוע הנדרש על המעבד.
- $int\ remaining_time$ זמן שנותר לביצוע (רלוונטי לאלגוריתמים עם הפקעה).
- $int\ is_completed$ – משתנה עזר, המסמן תהליכים שסיימו את ריצתם.
- $int\ waiting_time$, $int\ turnaround_time$, $int\ finish_time$ לצורך חישובי הסטטיסטיקה בסיום.

דגשים והסברים (למידה עצמאית ודוגמאות)

מתוך חומר ההרצאה, נתמקד במושגים המרכזיים שיופיעו במשימות:

קריטריונים למדידה

- **Turnaround Time (TAT)** הזמן הכולל מרגע הגעת התהליך ועד סיומו.$(Finish - Arrival)$
- **Waiting Time (WT)** הזמן הכולל שהתהליך שהה בתור המוכנים $(TAT - Burst)$

אלגוריתם FCFS (First-Come, First-Served)

האלגוריתם הפשוט ביותר. מי שמגיע ראשון מבוצע ראשון עד הסוף (ללא הפקעה). החיסרון המרכזי הוא "אפקט השיירה" שבו תהליכים קצרים מחכים מאחורי תהליך ארוך מאוד.

אלגוריתם Round Robin (RR)

אלגוריתם עם הפקעה המבוסס על "פרוסת זמן" כל תהליך מקבל את המעבד לזמן קצוב. אם לא סיים, הוא חוזר לסוף התור.

- **חשוב :**אם פרוסת הזמן גדולה מאוד RR, מתנהג כמו FCFS. אם היא קטנה מדי, התקורה של החלפת הקשר (Context Switch) תפגע בביצועים.

תזמון SJF (Shortest Job First)

אלגוריתם **ללא הפקעה.** כאשר המעבד מתפנה, המערכת בוחרת מתוך תור המוכנים את התהליך שזמן הביצוע שלו ($Burst\ Time$) הוא הקצר ביותר. ברגע שתהליך התחיל לרוץ, הוא ימשיך עד לסיומו המלא, גם אם בזמן ריצתו הגיע למערכת תהליך קצר ממנו.

תזמון SRT (Shortest Remaining Time)

גרסה עם הפקעה של SJF (Shortest Job First) בכל פעם שמגיע תהליך חדש, המערכת בודקת האם זמן הביצוע שנותר לו קצר יותר מזה של התהליך הרץ כרגע. אם כן – מתבצעת הפקעה.

עקרונות בניית סימולטור

סימולטור תזמון הוא תוכנית המנהלת ציר זמן וירטואלי .בניגוד לתוכנית רגילה שרצה בזמן אמת, כאן אנחנו "משחקים" בתפקיד מערכת ההפעלה ומחליטים מה קורה בכל רגע.

ניהול הזיכרון והנתונים

לכל תהליך בסימולציה יש מצב משתנה. חשוב להבין שחלק מהשדות ב $struct$ הם נתוני קלט $(Arrival, Burst)$ וחלקם הם שדות עזר לחישוב $(Remaining\ Time, Finish\ Time)$

- שימו לב, לפני כל הרצה של אלגוריתם, חובה לאתחל מחדש את השדה $remaining_time$ לערך השדה $burst_time$ המקורי, אחרת לא תוכלו להריץ השוואה בין אלגוריתמים שונים על אותו מערך.

לולאת הזמן המרכזית

הסימולציה תתבצע בתוך לולאת $while$ שתרוץ עד שכל התהליכים יסיימו. בתוך הלולאה ישנם שלושה שלבים קריטיים:

1. **זיהוי תהליכים מוכנים (Ready Pool)** - בכל נקודת זמן T, רק תהליכים שמתקיים עבורם,

$arrival_time \leq T$ רלוונטיים להחלטה. תהליך שהגיע בזמן $T = 5$ לא יכול להיבחר אם השעון שלנו כרגע בזמן $T = 2$

2. **לוגיקת הבחירה -** זהו "הלב" של כל משימה. כאן אתם שואלים: "מבין כל התהליכים שזמינים כרגע ולא סיימו, מי הבא בתור."?
   - בתזמון $FCFS$ – התהליך שהגיע קודם.
   - בתזמונים SJF/SRT – התהליך עם הזמן הקצר ביותר.
   - בתזמון RR – התהליך שנמצא בראש התור.
3. **קידום השעון**
   - באלגוריתם ללא הפקעה, ברגע שבחרתם תהליך, אתם יכולים "לקפוץ" קדימה בזמן $current_time\ +=\ burst_time$
   - באלגוריתם עם הפקעה, אסור לקפוץ! עליכם לקדם את השעון ביחידה אחת $current_time++$ להוריד 1 מהשדה $remaining_time$ של התהליך הנבחר, ולחזור לתחילת הלולאה כדי לבדוק אם הגיע תהליך "טוב יותר" ברגע זה.

טיפול בזמן "מת" $(Idle\ CPU)$

נמה קורה אם $current_time\ =\ 10$ אבל התהליך הבא מגיע רק בזמן $T = 15$? אם חיפשתם תהליך ולא מצאתם אף אחד שמוכן (למרות שהסימולציה עוד לא נגמרה), עליכם פשוט לקדם את השעון ($current_time$) בלי לבצע שום תהליך. <u>זהו זמן אי-עשייה של המעבד.</u>

חישוב מדדי הביצוע

ברגע שתהליך סיים ($remaining = 0$), עליכם לנעול את הנתונים שלו:

1. $finish_time = current_time$
2. $turnaround_time = finish_time - arrival_time$
3. $waiting_time = turnaround_time - burst_time$

מה זה דורש מכם במשימות?

בכל משימה תצטרכו לממש פונקציה שמקבלת את המערך ומבצעת עליו את הלולאה הזו לפי חוקי האלגוריתם הספציפי.

משימות לביצוע

עליכם לממש 4 פונקציות תזמון שונות. כל פונקציה מקבלת מערך של $Process$ ואת גודלו, ומעדכנת בתוכו את זמני הסיום וההמתנה.

משימה 1: FCFS (First-Come First-Served)

האלגוריתם הפשוט ביותר. ביצוע לפי סדר ההגעה בלבד, ללא הפקעות.

משימה 2: SJF (Shortest Job First)

אלגוריתם **ללא הפקעה** .בכל פעם שהמעבד מתפנה, בחרו את התהליך עם ה $burst_time$ הקצר ביותר מבין המוכנים.

משימה 3: Round Robin (RR)

אלגוריתם **עם הפקעה** .לכל תהליך מוקצבת פרוסת זמן ($Q = 2$) אם התהליך לא סיים, הוא חוזר לסוף התור.

משימה 4: SRT (Shortest Remaining Time)

הגרסה **עם הפקעה** של SJF . בכל יחידת זמן, בחרו את התהליך בעל ה $remaining_time$ המינימלי. אם הגיע תהליך קצר יותר מהנוכחי, בצעו החלפה.

משימה 5: השוואת ביצועים

במשימה זו עליכם להשתמש בקובץ ה $main$ המצורף:

1. הקובץ קורא נתוני תהליכים מקובץ טקסט $input.txt$
2. הוא מריץ את כל 4 הפונקציות שכתבתם על אותם נתונים.
3. הוא מדפיס טבלת השוואה של **זמן המתנה ממוצע** לכל אלגוריתם.

- **דרישה:** עליכם לוודא שהפונקציות שלכם לא "דורסות" את נתוני המקור (השתמשו בהעתק של המערך לכל ריצה).

מה להגיש ובאיזה פורמט

יש להגיש קובץ ZIP בשם $Lab4_ID.zip$ (החליפו את ID בתעודת הזהות שלכם) על הקובץ להכיל את הקבצים:

1. $scheduler_lib.c$ המכיל את כל 4 המימושים.
2. $scheduler_lib.h$ קובץ *Header* עם הגדרת ה $stuct$ והפונקציות
3. $main_compare.c$
4. $input.txt$ – קובץ נתוני התהליכים.

5. $Makefile$ המייצר את תוכנית ההשוואה.

מבנה קובץ הקלט $(input.txt)$

הקובץ בנוי בצורה המאפשרת למערכת לדעת מראש כמה זיכרון להקצות (עבור מערך התהליכים) ולאחר מכן את הפרמטרים של כל תהליך.

מבנה הקובץ:

1. **שורה ראשונה:** מספר שלם (N) המייצג את כמות התהליכים הכוללת בסימולציה.
2. **שורות הבאות:** עבור כל תהליך, מופיעים שני מספרים מופרדים ברווח:
   - **מספר ראשון:** זמן הגעה$(arrival_time)$
   - **מספר שני:** זמן ביצוע $(burst_time)$

<u>דגשים להגשה:</u>

- יש לוודא שלכל הקבצים יש הרשאות ריצה.(chmod +x)
- הבודק האוטומטי משווה פלטים בצורה מדויקת .(Case Sensitive) אין להדפיס הודעות נוספות מעבר לנדרש.
- וודאו שכל סקריפט מתמודד נכון עם ארגומנטים חסרים.

אי הקפדה על הנחיות, כולל פורמט הגשה הדיגיטלי, תגרור הורדה בציון התרגיל.